From AI Script to Production API

FastAPI + Uvicorn

Turn your LangChain agent into a real service the world can use

A Hands-On 3-Hour Workshop

Know Your Instructor

Varush H.

Senior Data Scientist · PwC India

Senior Data Scientist @ PwC India

Leads high-impact AI/ML projects delivering measurable business ROI across enterprise clients in consulting.

Agentic & Generative AI Practitioner

Architected and shipped production-grade Agentic AI, NLP, Deep Learning, and LLM-powered solutions.

Student Mentor & Educator

Mentors aspiring data scientists and junior engineers — guiding careers, projects, and skill-building in AI/ML.

Workshop Facilitator — AI

Delivers hands-on workshops translating complex AI concepts into practical, build-ready knowledge for learners.

Scan to Connect on in

Where You Are & Where You're Going

Last Session

You built an AI agent that runs in a Python script or notebook. It works — but only on YOUR machine, when YOU run it manually.

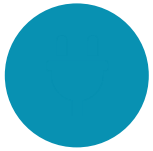
After Today

Your agent is a live API that any app, website, or teammate can call over the internet — 24/7, no manual running needed.

Today's Outcome

You will be able to convert ANY AI script into a production-ready API — a skill every AI engineer needs.

Session Agenda — 3-Hour Roadmap



Hour 1

API Foundations (Practical)

What an API really is, HTTP, requests/responses, JSON, REST — explained with everyday analogies

Hour 2

Building with FastAPI

Your first endpoint, path & query params, Pydantic models, auto-docs, error handling

Hour 3

Serve LangChain + Uvicorn

Wrap your AI agent in an API, streaming responses, run with Uvicorn, go to production

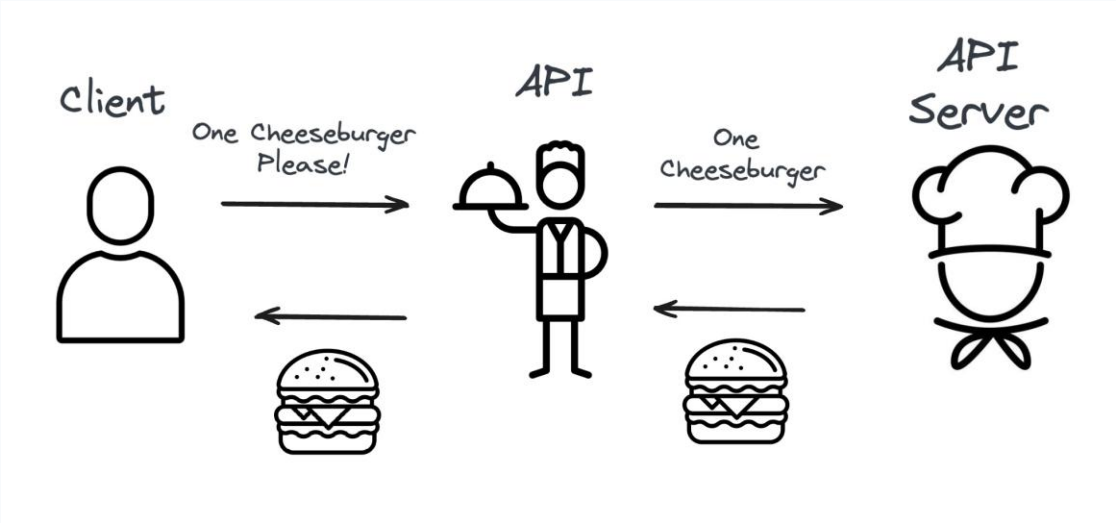
Hour 1: API Foundations

Understanding APIs the Practical Way — No Jargon

60 min

What is an API? The Restaurant Analogy

Think of ordering food at a restaurant. You don't walk into the kitchen — you use a waiter.



An API is a contract: "Send me a request in this format, and I'll send back a response in that format." You never see the kitchen.

You Already Use APIs Every Day

Weather App

Calls a weather API to fetch today's forecast

Login with Google

Uses Google's API to verify who you are

Online Payment

Calls Stripe/PayPal API to process your card

Maps in Uber

Uses Google Maps API for directions

ChatGPT App

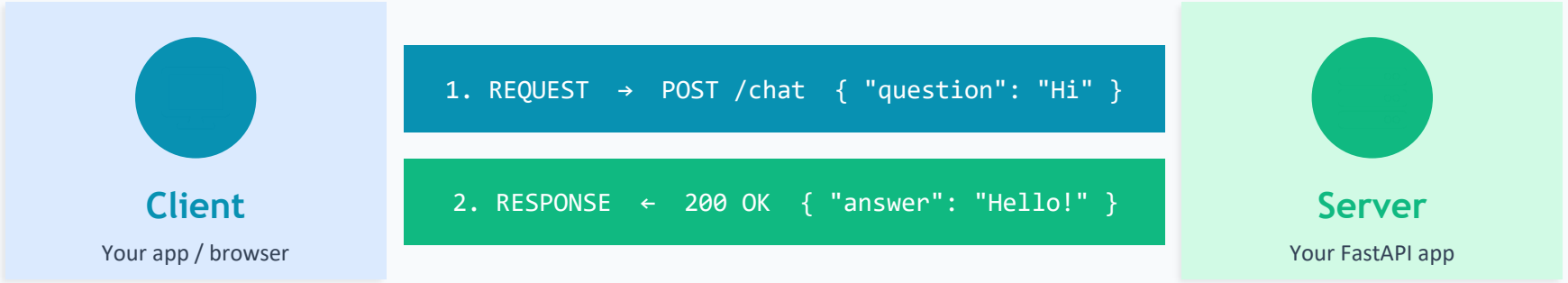
Talks to OpenAI's API behind the scenes

Social Media Share

Posts via the platform's API

APIs are everywhere. Today, YOU build one.

How an API Call Works: Request & Response



A Request Contains:

Method:	GET, POST, PUT, DELETE
URL/Endpoint:	/chat, /agent/run
Headers:	auth tokens, content-type
Body:	the data (JSON)

A Response Contains:

Status Code:	200 OK, 404, 500
Headers:	content-type, length
Body:	the result (JSON)

HTTP Methods — The Verbs of APIs

GET	**Retrieve data**	`GET /agents → list all agents`	*Reading a menu*
POST	**Create / send data**	`POST /chat → send a question`	*Placing an order*
PUT	**Update existing data**	`PUT /agent/1 → update config`	*Changing your order*
DELETE	**Remove data**	`DELETE /session/5 → end chat`	*Canceling an order*

For serving AI agents, you'll mostly use POST — sending a question and getting an answer back.

Status Codes — What the Server Tells You

2xx

Success

Everything worked as expected

`200 OK · 201 Created`

4xx

Client Error

You (the caller) made a mistake

`400 Bad Request · 404 Not Found · 401 Unauthorized`

5xx

Server Error

The server broke while processing

`500 Internal Error · 503 Unavailable`

JSON — The Language APIs Speak

What is JSON?

JSON (JavaScript Object Notation) is a simple, human-readable format for exchanging data. It's how the client and server pass information back and forth.

Key Rules:

- Data is in key–value pairs
- Keys are always in "quotes"
- Values: string, number, boolean, list, object
- Looks just like a Python dictionary!

Example: An AI Chat Request & Response

```
// Request body
{
  "question": "What is FastAPI?",
  "session_id": "user_123",
  "stream": false
}
```

```
// Response body
{
  "answer": "FastAPI is a modern,
            fast web framework...",
  "tokens_used": 42,
  "status": "success"
}
```

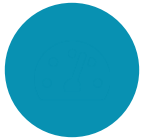
Hour 2: Building with FastAPI

From Zero to Your First Working Endpoint

60 min

What is FastAPI?

FastAPI is a modern, high-performance Python web framework for building APIs. It's designed to be fast to run AND fast to code — with automatic validation, documentation, and async support out of the box.



Fast

One of the fastest Python frameworks — on par with Node.js & Go

Easy to Learn

Write a working API in ~10 lines of clean Python code

Auto Validation

Pydantic checks incoming data types automatically

Auto Docs

Interactive API docs generated for free (Swagger UI)

Why FastAPI is Perfect for AI Apps

Async Support

AI calls (to LLMs) are slow & I/O-bound. Async lets your API handle many requests while waiting for the LLM to respond.

Streaming Responses

Stream tokens as the LLM generates them — just like ChatGPT's typing effect — instead of waiting for the full answer.

Type Safety

Pydantic models ensure the question, parameters, and response are always the right shape — fewer runtime bugs.

Production Ready

Used by Microsoft, Uber, Netflix. Scales from a prototype to millions of requests with the same code.

Setting Up FastAPI

Installation

```
# Create & activate virtual environment
python -m venv venv
source venv/bin/activate   # Windows: venv\Scripts\activate

# Install FastAPI + Uvicorn server
pip install fastapi uvicorn

# For our AI app later:
pip install langchain langchain-openai
```

Two Packages

`fastapi`

The framework — defines your API

`uvicorn`

The server — actually runs it

Project Structure

```
my-ai-api/
├── main.py          # Your FastAPI app lives here
├── agent.py         # Your LangChain agent code
├── .env             # API keys (OPENAI_API_KEY=...)
└── requirements.txt # Dependencies
```

Your First FastAPI Endpoint

```python
# main.py
from fastapi import FastAPI

# 1. Create the app
app = FastAPI()

# 2. Define an endpoint with a decorator
@app.get("/")
def read_root():
    return {"message": "Hello, API!"}

# 3. Add another endpoint
@app.get("/health")
def health_check():
    return {"status": "healthy"}

# That's it! A working API.
# Run it with:
#   uvicorn main:app --reload
```

Breaking It Down

`@app.get("/")`

Registers a GET endpoint at this URL

`def read_root()`

The function that runs on each call

`return {...}`

FastAPI auto-converts to JSON

Try It in the Browser

Once running, visit:

`localhost:8000`

→ `{"message": "Hello, API!"}`

And docs at:

`localhost:8000/docs`

Passing Data: Path & Query Parameters

Path Parameters

Part of the URL path. Used to identify a specific resource.

```
@app.get("/agent/{agent_id}")
def get_agent(agent_id: int):
    return {"id": agent_id}

# Call: GET /agent/42
# → {"id": 42}
```

FastAPI auto-converts agent_id to int and validates it!

Query Parameters

After the ? in a URL. Used for filtering, options, search.

```
@app.get("/search")
def search(q: str, limit: int = 10):
    return {"query": q,
            "limit": limit}

# Call: GET /search?q=ai&limit=5
# → {"query": "ai", "limit": 5}
```

limit has a default (10), so it's optional. q is required.

Request Body & Pydantic Models

```python
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# Define the shape of incoming data
class ChatRequest(BaseModel):
    question: str
    session_id: str = "default"
    temperature: float = 0.7

# Use it in a POST endpoint
@app.post("/chat")
def chat(request: ChatRequest):
    return {
        "received": request.question,
        "session": request.session_id
    }
```

Why Pydantic?

- Validates types automatically
- Rejects bad data with clear errors
- Sets defaults for optional fields
- Documents your API shape

Auto Validation Example

If a client sends:

```
{ "temperature": "hot" }
```

FastAPI auto-rejects with 422:

```
"temperature must be a number"
```

No code needed from you!

Automatic Interactive Docs (Swagger UI)

FastAPI automatically generates interactive API documentation. Visit /docs and you get a full UI to explore and TEST your endpoints — no Postman needed. This comes free, just from your code.

See All Endpoints

Every route, method, and parameter listed automatically

Test Live

Click "Try it out", enter data, hit Execute — see real responses

Share with Team

Frontend devs & testers explore your API without asking you

```
Two free doc UIs:   /docs  (Swagger)   ·   /redoc  (ReDoc)
```

Handling Errors Gracefully

```python
from fastapi import FastAPI, HTTPException

app = FastAPI()

@app.post("/chat")
def chat(request: ChatRequest):
    # Validate business logic
    if len(request.question) > 1000:
        raise HTTPException(
            status_code=400,
            detail="Question too long (max 1000)"
        )

    try:
        answer = run_agent(request.question)
        return {"answer": answer}
    except Exception as e:
        raise HTTPException(
            status_code=500,
            detail=f"Agent failed: {str(e)}"
        )
```

Good Error Handling

Return clear status codes

400 for bad input, 500 for failures

Give helpful messages

Tell the caller what went wrong

Never leak secrets

Don't expose stack traces or keys

Wrap risky calls

Try/except around LLM & tool calls

Hour 3: Serve LangChain + Uvicorn

Wrap Your AI Agent in an API & Ship It

60 min

The Goal: Wrap Your LangChain Agent in an API

BEFORE — A Script

```
# agent.py
agent = create_agent(...)

result = agent.invoke(
  "What is FastAPI?"
)
print(result)

# Only runs when YOU
# run python agent.py
```

AFTER — An API

```
# main.py
@app.post("/chat")
def chat(req: ChatRequest):
  result = agent.invoke(
    req.question
  )
  return {"answer": result}

# Anyone can call it!
```

The Key Idea

Your agent logic doesn't change. You simply wrap the agent.invoke() call inside a FastAPI endpoint. The endpoint receives a question over HTTP, passes it to your agent, and returns the answer as JSON. That's the whole trick.

Serving a LangChain Agent — Full Code

```python
# main.py — Complete AI Agent API
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub

app = FastAPI(title="My AI Agent API")

# ---- Set up the agent once, at startup ----
llm = ChatOpenAI(model="gpt-4", temperature=0)
tools = [...]   # your tools from last session
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, max_iterations=10)

# ---- Define the request shape ----
class ChatRequest(BaseModel):
    question: str

# ---- The endpoint that serves the agent ----
@app.post("/chat")
def chat(request: ChatRequest):
    try:
        result = executor.invoke({"input": request.question})
        return {"answer": result["output"], "status": "success"}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Breaking Down the Agent API

1 Initialize Once

Create the LLM, tools, and agent at startup — NOT inside the endpoint. This avoids rebuilding the agent on every request (slow & wasteful).

2 Define Request Shape

ChatRequest(BaseModel) declares that every call must include a 'question' string. FastAPI validates this automatically.

3 Wrap invoke() in Endpoint

The /chat POST endpoint receives the question, passes it to executor.invoke(), and returns the agent's output as JSON.

4 Handle Failures

Wrap the agent call in try/except. If the agent errors, return a clean 500 response instead of crashing the server.

What is Uvicorn?

Uvicorn is the server that actually runs your FastAPI app. FastAPI defines WHAT your API does; Uvicorn handles the live network connections — listening for requests, passing them to FastAPI, and sending responses back. It's an ASGI server built for async, high-performance Python.

The Analogy

FastAPI = the restaurant's recipes & menu (the logic).

Uvicorn = the building, the doors, and the staff that actually open up, let customers in, and keep things running.

You need both: recipes alone don't serve customers.

Key Facts

ASGI server
Handles async — perfect for AI's slow LLM calls

Lightning fast
Built on uvloop & httptools

Auto-reload
Restarts on code changes during dev

Production-grade
Scales with multiple workers

Running Your API with Uvicorn

Development

```
uvicorn main:app --reload
#       ^^^^ ^^^  ^^^^^^^^
#       file app  auto-restart when code changes (dev only)
```

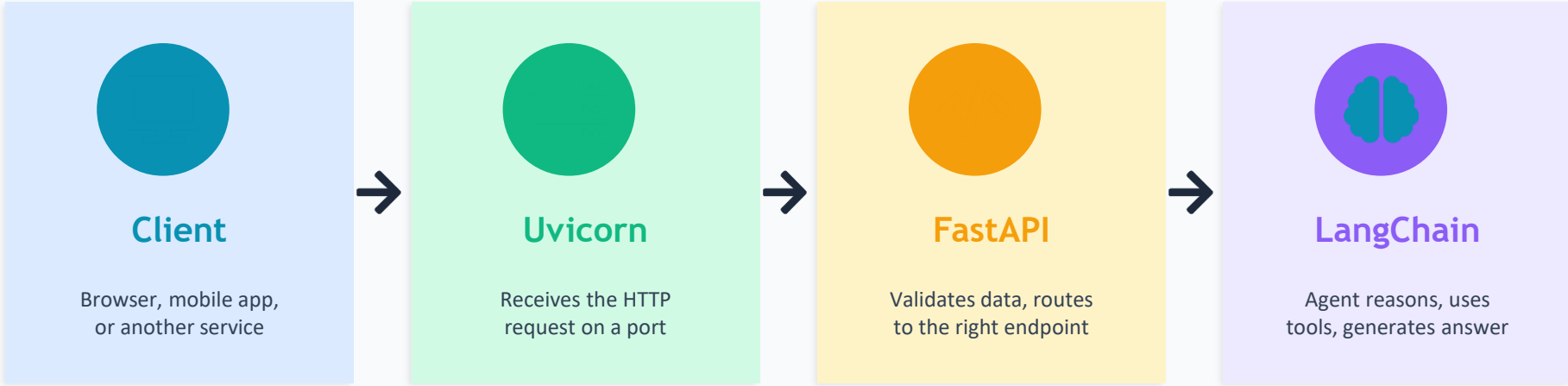
Decoding the command

main = main.py file · app = the FastAPI() object inside it · --reload = restart on save

Production

```
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
#                listen to all IPs    set port   run 4 parallel processes
```

The Full Picture: How It All Connects



Request flows left → right, response flows right → left

The question travels Client → Uvicorn → FastAPI → LangChain agent. The answer travels back the same path as JSON. Each layer has one clear job.

Going to Production — What Else Matters

Environment Variables

Store API keys in .env, never in code. Load with python-dotenv.

CORS

Allow your frontend's domain to call the API (CORSMiddleware).

Rate Limiting

Cap requests per user to prevent abuse and runaway LLM costs.

Authentication

Protect endpoints with API keys or OAuth so only authorized users call.

Logging & Monitoring

Track requests, errors, latency, and token usage in production.

Deployment

Containerize with Docker, deploy to Render, Railway, AWS, or Fly.io.

Quick Wins: Env Variables & CORS

Loading API Keys Safely

```
# .env file
OPENAI_API_KEY=sk-...

# main.py
from dotenv import load_dotenv
load_dotenv()   # reads .env

# Now os.environ has your key
# LangChain picks it up
# automatically
```

Allowing Your Frontend (CORS)

```
from fastapi.middleware.cors \
    import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://myapp.com"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Why These Two First?

Env variables keep your secret keys out of your code (and out of GitHub). CORS is the #1 reason a frontend can't talk to your API — without it, browsers block the request. Set both up early and you avoid the two most common beginner roadblocks.

Structuring Your Agent API

Controller · Service · Generator · Domain · System

Organizing a production FastAPI codebase the professional way

Part 3 — Deep Dive

The Problem: Everything in One main.py

What students usually write:

```
# main.py — 200 lines & growing
app = FastAPI()

llm = ChatOpenAI(...)        # setup
tools = [...]                # tools
agent = create_agent(...)    # agent

@app.post("/chat")
def chat(req):
    # validation
    # business rules
    # call the agent
    # save history
    # format response
    # ...all jammed here
    return {...}
```

Works for a demo — but becomes unmaintainable fast.

Why It Breaks Down

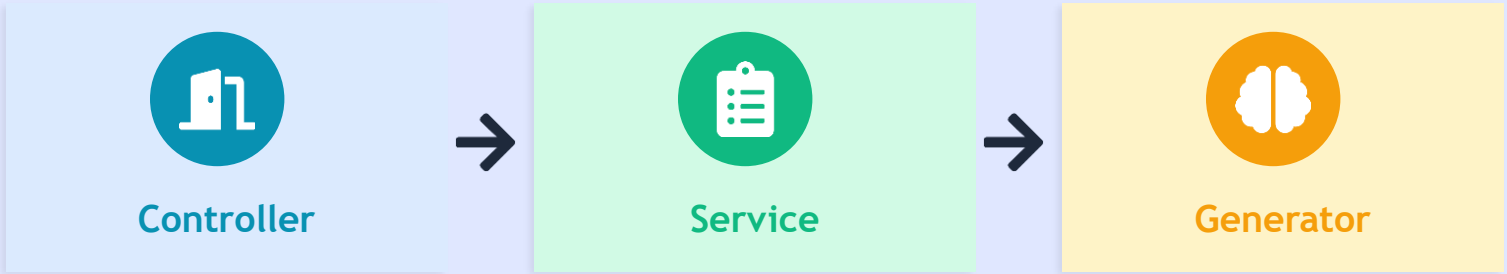
- Hard to find anything in one giant file
- Can't test the agent without the API
- Changing one thing risks breaking others
- Two developers can't work without conflicts
- Business logic tangled with HTTP code
- Impossible to reuse the agent elsewhere

The Solution: 5 Clean Layers

Controller	Handles HTTP. Receives requests, returns responses.	The receptionist
Service	Business logic. Orchestrates the whole flow.	The manager
Generator	Runs the AI agent. Produces the answer.	The expert worker
Domain	Data models & core entities. Shared everywhere.	The shared vocabulary
System	App setup, config, wiring, lifecycle.	The building & utilities

How the Layers Connect

SYSTEM — wires everything together, provides config & runtime



DOMAIN — shared data models (ChatRequest, ChatResponse) used by every layer

Request: Controller → Service → Generator. Response flows back the same way.
Domain models are the common language they all speak. System holds it all together.

1. Domain — The Shared Data Models

What It Is

The Domain layer defines the core data structures your app works with — as Pydantic models. They have NO logic, just shape. Every other layer speaks in these models.

Why Separate It?

- One source of truth for data shapes
- Auto-validation via Pydantic
- Reused by Controller, Service & Generator

```python
# domain/models.py
from pydantic import BaseModel
from typing import Optional

class ChatRequest(BaseModel):
    """What comes IN."""
    question: str
    session_id: str = "default"
    temperature: float = 0.7

class ChatResponse(BaseModel):
    """What goes OUT."""
    answer: str
    session_id: str
    tokens_used: int

class AgentConfig(BaseModel):
    """Core entity."""
    model: str = "gpt-4"
    max_iterations: int = 10

# Pure data. No logic. No HTTP.
# No LangChain. Just shapes.
```

2. Generator — The AI Engine

What It Is

The Generator owns the LangChain agent. Its ONLY job is to take a question and generate an answer. It knows nothing about HTTP, databases, or business rules — pure AI generation.

Why Separate It?

- Swap GPT-4 → Claude without touching the API
- Test the agent standalone, no server needed
- Reuse the same agent in a CLI, worker, or API

```python
# generator/agent_generator.py
from langchain_openai import ChatOpenAI
from langchain.agents import (
    create_react_agent, AgentExecutor)
from langchain import hub
from domain.models import AgentConfig

class AgentGenerator:
    """Owns the agent. Only generates."""
    def __init__(self, tools, config:
                 AgentConfig):
        llm = ChatOpenAI(
            model=config.model,
            temperature=0)
        prompt = hub.pull("hwchase17/react")
        agent = create_react_agent(
            llm, tools, prompt)
        self.executor = AgentExecutor(
            agent=agent, tools=tools,
            max_iterations=config.max_iterations)

    def generate(self, question: str) -> str:
        result = self.executor.invoke(
            {"input": question})
        return result["output"]
```

3. Service — The Business Logic

What It Is

The Service orchestrates the flow. It applies business rules, calls the Generator, saves history, handles retries — the 'what should happen' logic. It's the brain of the operation (not the AI brain — the workflow brain).

Why Separate It?

- Business rules live in ONE place
- Controller stays thin & simple
- Logic testable without HTTP or AI

```
# service/chat_service.py
from domain.models import (
    ChatRequest, ChatResponse)
from generator.agent_generator import (
    AgentGenerator)

class ChatService:
    """Orchestrates business logic."""
    def __init__(self, generator:
                 AgentGenerator):
        self.generator = generator

    def handle_chat(self, request:
              ChatRequest) -> ChatResponse:
        # 1. business rule: guard length
        if len(request.question) > 1000:
            raise ValueError("Too long")
        # 2. delegate to the Generator
        answer = self.generator.generate(
            request.question)
        # 3. (could save history here)
        # 4. build the domain response
        return ChatResponse(
            answer=answer,
            session_id=request.session_id,
            tokens_used=len(answer.split()))
```

4. Controller — The HTTP Doorway

What It Is

The Controller is the thin HTTP layer — your FastAPI routes. It receives the request, hands it to the Service, and returns the response. That's it. No business logic, no AI code — just the doorway.

Why Keep It Thin?

- Easy to read — see all endpoints at a glance
- Swap REST for GraphQL without logic changes
- Maps HTTP errors to the right status codes

```python
# controllers/chat_controller.py
from fastapi import (
    APIRouter, Depends, HTTPException)
from domain.models import (
    ChatRequest, ChatResponse)
from service.chat_service import ChatService
from system.dependencies import get_service

router = APIRouter()

@router.post("/chat",
             response_model=ChatResponse)
def chat(
    request: ChatRequest,
    service: ChatService = Depends(get_service)
):
    try:
        return service.handle_chat(request)
    except ValueError as e:
        raise HTTPException(400, str(e))
    except Exception as e:
        raise HTTPException(500, str(e))

# Thin! Just receive → delegate → return.
```

5. System — The Wiring & Setup

What It Is

The System layer creates the FastAPI app, loads config, sets up CORS, wires dependencies together (dependency injection), and manages startup/shutdown. It's the infrastructure that holds everything together.

What Lives Here

- The FastAPI() app instance & routers
- Config & env variables (settings)
- Dependency injection (get_service)

```
# system/main.py
from fastapi import FastAPI
from controllers.chat_controller import router

app = FastAPI(title="AI Agent API")
app.include_router(router)

# system/dependencies.py  (wiring)
from generator.agent_generator import (
    AgentGenerator)
from service.chat_service import ChatService
from domain.models import AgentConfig
from my_tools import tools

# Build once, inject everywhere
_generator = AgentGenerator(
    tools, AgentConfig())
_service = ChatService(_generator)

def get_service() -> ChatService:
    return _service   # FastAPI injects this
```

A Request's Journey Through All 5 Layers

1	System	App starts. Generator & Service are built once and wired up via dependency injection.
2	Controller	POST /chat arrives. FastAPI validates it into a Domain ChatRequest model.
3	Service	Controller calls service.handle_chat(). Service applies business rules.
4	Generator	Service calls generator.generate(). The LangChain agent reasons & answers.
5	Domain	Service wraps the answer in a ChatResponse model and returns it up the chain.
6	Controller	Returns the ChatResponse as JSON. FastAPI sends HTTP 200 to the client.

The Project Structure It Creates

```
my-agent-api/
│
├── domain/
│   └── schemas.py          # ChatRequest, ChatResponse
│
├── generator/
│   └── agent_generator.py # the LangChain agent
│
├── service/
│   └── chat_service.py    # business logic
│
├── controllers/
│   └── chat_controller.py # FastAPI routes
│
├── system/
│   ├── main.py            # FastAPI app
│   ├── config.py          # settings & env
│
├── .env                   # API keys
└── requirements.txt
```

Each Folder = One Job

`domain/`	Data shapes
`generator/`	AI agent
`service/`	Logic
`controllers/`	HTTP routes
`system/`	App & wiring

New dev finds anything in seconds.

Why This Matters — Before vs After

Concern	One Big main.py	5 Clean Layers
Find code	Scroll through 200+ lines	**Go to the right folder instantly**
Testing	Must spin up the whole API	**Test each layer in isolation**
Swap the LLM	Hunt through tangled code	**Change only the Generator**
Team work	Constant merge conflicts	**Each dev owns a layer**
Reuse agent	Copy-paste everywhere	**Import the Generator anywhere**
Onboarding	"Where is everything?"	**Structure explains itself**

Same code, organized into layers — but now it's testable, scalable, and team-ready. That's professional engineering.

One Rule to Remember

Each layer should have exactly ONE reason to change.

Controller	Service	Generator	Domain	System
HTTP changes	Rules change	AI changes	Data changes	Setup changes

You Did It!

You can now turn any AI script into a production API

Questions & Discussion

Now go ship your agent to the world! 🚀